

Estudio de la clase Stack en Java

Raúl Ramírez Domínguez
1ºDAM

ÍNDICE

1. INTRODUCCIÓN

1.1. Contexto del trabajo

1.2. Objetivo del estudio

2. LA ESTRUCTURA DE DATOS PILA

2.1. Definición de pila

2.2. Funcionamiento LIFO (Last In, First Out)

2.3. Ejemplos de uso en programación

3. LA CLASE STACK EN JAVA

3.1. Definición de Stack

3.2. Ubicación en el API de Java (java.util)

3.3. Jerarquía de clases (relación con Vector)

3.4. Características principales

3.5. Creación y uso básico de una pila

4. MÉTODOS PRINCIPALES DE STACK

4.1. Método push(E item)

4.2. Método pop()

4.3. Método peek()

4.4. Método empty()

4.5. Método search(Object o)

4.6. Métodos heredados de Vector

4.6.1. size()

4.6.2. isEmpty()

4.6.3. clear()

4.6.4. get(int index)

5. VENTAJAS Y DESVENTAJAS DE STACK

5.1. Ventajas

5.2. Desventajas

5.3. Posibles dificultades al utilizar Stack

6. COMPARACIÓN CON ALTERNATIVAS MODERNAS

6.1. La interfaz Deque

6.2. Uso de ArrayDeque como pila

6.3. Diferencias entre Stack y Deque

6.4. Evolución en las versiones de Java

7. CASO PRÁCTICO: SISTEMA DE INVENTARIO CON DESHACER ACCIONES

7.1. Enunciado del problema

7.2. Justificación del uso de Stack

7.3. Descripción general del sistema

8. IMPLEMENTACIÓN EN ECLIPSE

8.1. Estructura del proyecto

8.2. Clases del modelo

8.2.1. Clase Producto

8.2.2. Clase AccionInventario

8.3. Capa de acceso a datos (CRUD)

8.3.1. Clase InventarioDAO

8.4. Lógica de negocio

8.4.1. Clase GestorInventario

8.4.2. Uso de Stack en la gestión de acciones

8.5. Interfaz de usuario

8.5.1. Clase Main

8.5.2. Menú de opciones

9. CONCLUSIÓN

10. BIBLIOGRAFÍA / WEBGRAFÍA

2. LA ESTRUCTURA DE DATOS PILA

2.1. Definición de pila

Una pila es una estructura de datos lineal en la que los elementos se organizan de forma que solo se puede acceder a uno de sus extremos, denominado cima. Todas las operaciones de inserción y eliminación se realizan en ese mismo punto.

Este tipo de estructura es especialmente útil cuando interesa trabajar con los elementos en orden inverso al que se han introducido.

2.2. Funcionamiento LIFO (Last In, First Out)

El funcionamiento de una pila se basa en el principio LIFO (*Last In, First Out*), que significa “el último en entrar es el primero en salir”.

Esto implica que cada vez que se añade un nuevo elemento, este se coloca en la cima, y cuando se elimina un elemento, siempre se retira el que está en esa posición.

Este comportamiento hace que la pila sea muy adecuada para gestionar procesos en los que es necesario revertir acciones o trabajar con el elemento más reciente.

2.3. Ejemplos de uso en programación

Las pilas aparecen en muchos contextos dentro de la programación. Algunos ejemplos habituales son:

- Sistemas de deshacer y rehacer acciones, como en editores de texto
- Gestión del historial de navegación en aplicaciones
- Evaluación de expresiones matemáticas
- Control de llamadas a funciones en la ejecución de programas

En todos estos casos, el uso de una pila permite gestionar la información de forma sencilla siguiendo el orden inverso al de inserción.

3. LA CLASE STACK EN JAVA

3.1. Definición de Stack

La clase Stack es una clase incluida en el paquete java.util que permite trabajar con una estructura de datos tipo pila en Java. Su funcionamiento sigue el modelo LIFO (Last In, First Out), por lo que el último elemento que se inserta es el primero en eliminarse.

Se trata de una clase genérica, lo que significa que puede almacenar cualquier tipo de objeto. Por ejemplo, se puede crear una pila de enteros, de cadenas de texto o de cualquier otra clase definida por el usuario.

3.2. Ubicación en el API de Java (java.util)

Stack forma parte del paquete java.util, que incluye la mayoría de las estructuras de datos utilizadas en Java, como listas, conjuntos o mapas.

Para poder utilizarla en un programa, es necesario importarla previamente:

```
import java.util.Stack;
```

Una vez importada, se puede crear una pila indicando el tipo de dato que va a contener:

```
Stack<String> pila = new Stack<>();
```

3.3. Jerarquía de clases (relación con Vector)

Una característica importante de Stack es que no es una clase independiente, sino que se hereda de otra clase llamada Vector.

La jerarquía es la siguiente:

Object → AbstractCollection → ArrayList → Vector → Stack

Esto implica que Stack no solo dispone de sus propios métodos, sino también de todos los métodos heredados de Vector, como añadir elementos, recorrer la colección o acceder por posición.

Sin embargo, esta herencia también tiene un inconveniente: permite realizar operaciones que no son propias de una pila, como acceder a un elemento por índice, lo que puede romper el comportamiento típico LIFO si no se usa correctamente.

3.4. Características principales

La clase Stack presenta las siguientes características:

- Sigue el modelo LIFO
- Permite insertar y eliminar elementos desde la cima
- Es una clase genérica
- Hereda métodos de Vector
- Puede lanzar excepciones si se accede a una pila vacía

Además, aunque sigue siendo funcional, actualmente se considera una clase más antigua dentro del API de Java, ya que existen alternativas más modernas para trabajar con pilas.

3.5. Creación y uso básico de una pila

Para utilizar una pila en Java, primero se crea una instancia de la clase Stack. A partir de ese momento, se pueden añadir y eliminar elementos utilizando sus métodos.

Ejemplo básico:

```
Stack<Integer> pila = new Stack<>();

pila.push(10);
pila.push(20);
pila.push(30);

System.out.println(pila.pop()); // devuelve 30
```

En este ejemplo se puede observar cómo el último elemento insertado (30) es el primero en salir, lo que demuestra el funcionamiento LIFO de la estructura.

4. MÉTODOS PRINCIPALES DE STACK

La clase Stack dispone de varios métodos que permiten trabajar con la estructura de pila de forma sencilla. A continuación se explican los más importantes.

4.1. Método push(E item)

El método push se utiliza para añadir un elemento a la pila. El elemento se inserta en la cima, pasando a ser el último elemento introducido.

Cada vez que se llama a este método, el nuevo elemento queda preparado para ser el primero en salir si posteriormente se utiliza pop.

Ejemplo:

```
Stack<Integer> pila = new Stack<>();
```

```
pila.push(10);
```

```
pila.push(20);
```

En este caso, el elemento 20 queda en la cima de la pila.

4.2. Método pop()

El método pop elimina el elemento que se encuentra en la cima de la pila y lo devuelve.

Si se intenta ejecutar este método cuando la pila está vacía, se produce una excepción (EmptyStackException). Por este motivo, es recomendable comprobar antes si la pila contiene elementos.

Ejemplo:

```
int valor = pila.pop(); // elimina y devuelve el último elemento
```

4.3. Método peek()

El método peek permite consultar el elemento que se encuentra en la cima de la pila sin eliminarlo.

Es útil cuando se necesita conocer el último elemento insertado, pero sin modificar la estructura de la pila.

Al igual que ocurre con pop, si la pila está vacía también se produce una excepción.

Ejemplo:

```
int valor = pila.peek(); // consulta el último elemento sin eliminarlo
```


4.4. Método empty()

Este método se utiliza para comprobar si la pila está vacía. Devuelve un valor booleano:

- true si la pila no contiene elementos
- false si contiene uno o más elementos

Suele utilizarse antes de realizar operaciones como pop o peek para evitar errores.

Ejemplo:

```
if (pila.empty()) {  
    System.out.println("La pila está vacía");  
}
```

4.5. Método search(Object o)

El método search permite buscar un elemento dentro de la pila.

Devuelve la posición del elemento contando desde la cima, empezando en 1. Si el elemento no se encuentra, devuelve -1.

Ejemplo:

```
int posicion = pila.search(20);
```

Si el elemento está en la cima, la posición será 1.

4.6. Métodos heredados de Vector

Además de los métodos propios de una pila, la clase Stack hereda otros métodos de la clase Vector, lo que amplía sus funcionalidades.

4.6.1. size()

Devuelve el número total de elementos que hay en la pila.

```
int total = pila.size();
```

4.6.2. isEmpty()

Indica si la pila está vacía, de forma similar a empty().

```
if (pila.isEmpty()) {  
    System.out.println("Sin elementos");  
}
```

4.6.3. clear()

Elimina todos los elementos de la pila.

```
pila.clear();
```

4.6.4. get(int index)

Permite acceder a un elemento concreto mediante su posición.

```
int valor = pila.get(0);
```

Sin embargo, este método no es propio de una pila tradicional, ya que permite acceder a elementos que no están en la cima.

5. VENTAJAS Y DESVENTAJAS DE STACK

5.1. Ventajas

La clase Stack presenta varias ventajas que hacen que sea útil, especialmente a nivel educativo y en determinados tipos de aplicaciones.

En primer lugar, su uso es sencillo e intuitivo. Los métodos que ofrece (push, pop, peek, etc.) representan directamente las operaciones típicas de una pila, lo que facilita su comprensión y utilización.

Además, permite implementar fácilmente estructuras LIFO sin necesidad de crear una clase desde cero. Esto resulta especialmente útil en situaciones como sistemas de deshacer acciones, historiales o gestión de procesos donde interesa trabajar con el último elemento añadido.

Otra ventaja es que forma parte del API estándar de Java, por lo que no es necesario añadir librerías externas para utilizarla.

5.2. Desventajas

A pesar de sus ventajas, Stack también presenta varias limitaciones que hacen que no sea la opción más recomendada en aplicaciones modernas.

Una de las principales desventajas es que hereda de la clase Vector, lo que implica que incluye muchos métodos que no son propios de una pila. Esto puede llevar a un uso incorrecto de la estructura, ya que se pueden realizar operaciones que rompen el comportamiento LIFO, como acceder directamente a elementos por posición.

Además, Vector está diseñado como una clase sincronizada, lo que puede afectar al rendimiento cuando no es necesario trabajar en entornos concurrentes.

Otra desventaja importante es que Stack se considera una clase antigua dentro del API de Java. En la práctica actual, se recomienda utilizar otras estructuras más modernas que ofrecen un diseño más adecuado y eficiente para trabajar como pilas.

5.3. Posibles dificultades al utilizar Stack

Al trabajar con Stack, pueden aparecer algunos problemas si no se utiliza correctamente.

Uno de los más comunes es intentar acceder a elementos cuando la pila está vacía. Métodos como pop o peek lanzan una excepción en este caso, por lo que es necesario comprobar previamente si la pila contiene elementos utilizando empty() o isEmpty().

Otra dificultad es el uso indebido de métodos heredados de Vector. Por ejemplo, acceder a elementos mediante índices puede hacer que el programa deje de comportarse como una pila real.

Por último, es importante tener en cuenta que, aunque Stack funciona correctamente, no siempre es la mejor elección en proyectos actuales, por lo que conviene conocer alternativas más adecuadas.

6. COMPARACIÓN CON ALTERNATIVAS MODERNAS

6.1. La interfaz Deque

En versiones más recientes de Java, se introdujo la interfaz Deque (Double Ended Queue), que permite trabajar con estructuras en las que se pueden insertar y eliminar elementos por ambos extremos.

Aunque su nombre hace referencia a una cola doble, Deque también puede utilizarse perfectamente como una pila, ya que ofrece métodos equivalentes a los de Stack, como push, pop y peek.

Esto hace que sea una alternativa más flexible, ya que permite trabajar tanto como pila (LIFO) como cola (FIFO), dependiendo de cómo se utilicen sus métodos.

6.2. Uso de ArrayDeque como pila

Una de las implementaciones más utilizadas de Deque es ArrayDeque. Esta clase permite trabajar con pilas de forma eficiente utilizando una estructura basada en arrays dinámicos.

En la práctica, ArrayDeque suele ser la opción recomendada cuando se necesita implementar una pila en Java.

Ejemplo de uso como pila:

```
Deque<Integer> pila = new ArrayDeque<>();
```

```
pila.push(10);
```

```
pila.push(20);
```

```
int valor = pila.pop();
```

Este código realiza las mismas operaciones que una Stack, pero utilizando una estructura más moderna.

6.3. Diferencias entre Stack y Deque

Existen varias diferencias importantes entre ambas opciones:

- Stack es una clase antigua que hereda de Vector, mientras que Deque es una interfaz más moderna y flexible.
- Stack incluye métodos heredados que no son propios de una pila, mientras que Deque ofrece una estructura más limpia y coherente.
- ArrayDeque, como implementación de Deque, suele tener mejor rendimiento que Stack en la mayoría de los casos.
- Deque permite trabajar tanto como pila como cola, mientras que Stack solo permite comportamiento LIFO.

6.4. Evolución en las versiones de Java

La clase Stack existe desde las primeras versiones de Java, lo que explica su diseño basado en Vector.

Sin embargo, con el paso del tiempo, el lenguaje ha ido incorporando nuevas interfaces y clases dentro del framework de colecciones, como Deque, que ofrecen soluciones más eficientes y mejor diseñadas.

Por este motivo, en el desarrollo actual se suele recomendar el uso de Deque y sus implementaciones, como ArrayDeque, en lugar de Stack, aunque esta última sigue siendo válida y funcional.

7. CASO PRÁCTICO: SISTEMA DE INVENTARIO CON DESHACER ACCIONES

7.1. Enunciado del problema

Se desea desarrollar una aplicación de consola en Java que permita gestionar el inventario de una tienda.

El sistema deberá ofrecer las siguientes funcionalidades:

- Dar de alta nuevos productos
- Eliminar productos existentes
- Modificar el stock de un producto
- Mostrar el listado de productos

Además, como funcionalidad adicional, el sistema permitirá deshacer la última acción realizada, de manera que el usuario pueda revertir cambios recientes en el inventario.

Para implementar esta funcionalidad se utilizará una pila (Stack), en la que se almacenarán las acciones realizadas por el usuario.

7.2. Justificación del uso de Stack

El uso de una pila en este sistema tiene sentido debido a la naturaleza de la funcionalidad de deshacer acciones.

Cada vez que el usuario realiza una operación sobre el inventario (alta, baja o modificación), dicha acción se guarda en la pila. De esta forma, la última acción realizada será la primera en poder deshacerse.

Este comportamiento sigue exactamente el modelo LIFO (Last In, First Out), que es el funcionamiento propio de una pila.

Por tanto, Stack resulta adecuada para este caso, ya que permite almacenar las acciones en orden y recuperarlas fácilmente cuando se desea revertir la última operación.

7.3. Descripción general del sistema

El sistema estará compuesto por varias clases que permitirán organizar el programa de forma estructurada.

Por un lado, se definirán clases de modelo, como Producto, que representará los datos de cada elemento del inventario, y AccionInventario, que permitirá almacenar la información necesaria para deshacer una operación.

Por otro lado, se incluirá una clase encargada de gestionar los datos (InventarioDAO), donde se implementarán las operaciones básicas de inserción, eliminación y consulta.

La lógica principal del sistema se desarrollará en una clase gestora (GestorInventario), que utiliza una pila (Stack) para almacenar el historial de acciones realizadas.

Finalmente, el programa contará con una clase principal (Main) que mostrará un menú por consola, permitiendo al usuario interactuar con el sistema de forma sencilla.

8. IMPLEMENTACIÓN EN ECLIPSE

8.1. Estructura del proyecto

Para que la aplicación esté bien organizada, el proyecto se dividirá en varios paquetes, separando los distintos elementos del programa según su función. Esta organización hace que el código sea más claro, más fácil de mantener y más parecido a la forma en la que se desarrollan aplicaciones reales.

La estructura propuesta es la siguiente:

```
src/  
├── modelo  
├── dao  
├── servicio  
└── principal
```

Cada paquete tendrá una función concreta:

- modelo: contendrá las clases que representan los datos del programa
- dao: incluirá la clase encargada de gestionar el almacenamiento de los productos
- servicio: contendrá la lógica del programa y el uso de la pila
- principal: incluirá la clase Main, desde la que se ejecutará el menú

8.2. Clases del modelo

8.2.1. Clase Producto

La clase Producto representará cada producto del inventario. Será una clase sencilla, encargada de almacenar la información básica de cada elemento.

Los atributos propuestos son:

- id
- nombre
- stock
- precio

Además, esta clase tendrá:

- constructor
- getters y setters
- método toString()

Su función será servir como modelo de datos para trabajar con los productos dentro del sistema.

8.2.2. Clase AccionInventario

La clase AccionInventario servirá para guardar información sobre las operaciones realizadas en el inventario, de forma que luego puedan deshacerse.

Cada vez que el usuario haga una acción importante, se almacenará un objeto de esta clase en la pila.

Los atributos recomendados son:

- tipoAccion
- productoAntes
- productoDespues
- descripcion

Con esta información será posible saber:

- qué acción se hizo
- sobre qué producto
- cuál era su estado antes
- cuál fue su estado después

Esta clase será clave para implementar la funcionalidad de deshacer.

8.3. Capa de acceso a datos (CRUD)

8.3.1. Clase InventarioDAO

La clase InventarioDAO se encargará de gestionar la colección de productos del inventario. En este proyecto no se utilizará base de datos, por lo que los productos se almacenarán en memoria, por ejemplo en un ArrayList.

Esta clase incluirá las operaciones básicas del CRUD:

- insertar producto
- buscar producto por id
- listar productos
- eliminar producto
- actualizar datos si fuera necesario

Su función será separar el almacenamiento de datos de la lógica del programa, haciendo que la aplicación quede más ordenada.

8.4. Lógica de negocio

8.4.1. Clase GestorInventario

La clase GestorInventario será la más importante del proyecto, ya que en ella se implementará la lógica principal del sistema.

Esta clase se encargará de:

- dar de alta productos
- eliminar productos
- modificar stock
- consultar productos
- deshacer la última acción realizada

Para ello utilizará:

- un objeto de tipo InventarioDAO
- una pila de tipo Stack<AccionInventario>

Aquí será donde realmente se pondrán en práctica los métodos de Stack.

8.4.2. Uso de Stack en la gestión de acciones

La pila servirá para almacenar el historial de acciones realizadas por el usuario.

Por ejemplo:

- si se da de alta un producto, esa acción se guarda en la pila
- si se elimina un producto, también se guarda
- si se modifica el stock, se registra igualmente

Después, cuando el usuario seleccione la opción de deshacer, el sistema recuperará la última acción almacenada en la pila y revertirá sus efectos.

De esta manera, la clase Stack tendrá una función real dentro del programa y no aparecerá simplemente como una prueba aislada de métodos.

8.5. Interfaz de usuario

8.5.1. Clase Main

La clase Main contendrá el método principal del programa y será el punto de entrada de la aplicación.

En ella se mostrará el enunciado del ejercicio al principio, tal como pide el profesor, y a continuación se ejecutará un menú con las distintas opciones del sistema.

También será la encargada de recoger los datos introducidos por el usuario mediante teclado y llamar a los métodos correspondientes del gestor.

8.5.2. Menú de opciones

El menú del programa incluirá opciones como las siguientes:

1. Alta de producto
2. Eliminar producto
3. Modificar stock
4. Listar productos
5. Ver última acción registrada
6. Deshacer última acción
7. Mostrar número de acciones almacenadas
8. Salir

9. CONCLUSIÓN

A lo largo de este trabajo se ha estudiado la clase Stack de Java, una estructura de datos que permite trabajar siguiendo el modelo LIFO, en el que el último elemento que entra es el primero en salir.

Se han analizado sus métodos principales, como push, pop y peek, que permiten insertar, eliminar y consultar elementos en la cima de la pila. Además, también se han visto otros métodos útiles como empty, size o search, que ayudan a gestionar mejor la estructura.

Por otro lado, se han identificado algunas limitaciones de la clase Stack, como su herencia de Vector, lo que hace que incluya métodos que no son propios de una pila. También se ha explicado que, aunque sigue siendo válida, en Java actual se suelen utilizar alternativas como Deque o ArrayDeque.

En la parte práctica se ha desarrollado un programa completo de gestión de inventario, en el que se ha aplicado la clase Stack para implementar una funcionalidad de deshacer acciones. Esto ha permitido ver de forma clara cómo se puede utilizar una pila en un caso real, más allá de ejemplos simples.

En conclusión, la clase Stack es una herramienta útil para entender y aplicar estructuras LIFO en Java, y su uso resulta especialmente adecuado en situaciones donde es necesario revertir acciones o trabajar con el elemento más reciente.

10. BIBLIOGRAFÍA Y WEBGRAFÍA

- Oracle. Java Platform SE Documentation.
<https://docs.oracle.com/javase/10/docs/api/>
- Oracle. Class Stack<E>.
<https://docs.oracle.com/javase/10/docs/api/java/util/Stack.html>
- Oracle. Interface Deque<E>.
<https://docs.oracle.com/javase/10/docs/api/java/util/Deque.html>
- Oracle. Class ArrayDeque<E>.
<https://docs.oracle.com/javase/10/docs/api/java/util/ArrayDeque.html>